

USP – ICMC – SSC

SSC0601 – Introdução à Ciência de Computação I – 2026 (ICC1 – Lab.)

Prof. F.Osorio – Turma 01 e Turma 02

EXERCÍCIO 02 – Extra Classe
Banco ACME Master-Chef

Semestre: 2026/1

Descrição do Trabalho na Web e Wiki: SSC-601-2026(fosorio)

Web: [http://wiki.icmc.usp.br/index.php/SSC-601-2026\(fosorio\)](http://wiki.icmc.usp.br/index.php/SSC-601-2026(fosorio))

E-Mail: fosorio @icmc.usp.br

DATA DESTE DOCUMENTO: 19 / 06 / 2026 - Versão 1.0 Revisão Rev.00 – Versão Inicial!

VER sempre o PDF com descrição completa e mais atualizada!

TRABALHO ENTREGUE VIA RUNCODES.ICMC (PDF no RunCodes SSC0601 Turma 01 e 02

ATENÇÃO - DATA LIMITE DE UPLOAD: ver no RUNCODES.ICMC ! 26.06.2026 8h

RUNCODES - Disciplina SS0601

Informações Importantes:

- A entrega é junto ao **RUNCODES.ICMC**, junto a disciplina **SSC0601 (upload do código)**.
- É muito importante seguir exatamente as especificações, para correção automática pelo RunCodes.
- Os trabalhos podem ser realizados e enviados **INDIVIDUALMENTE**.
- Exercício **EXTRA-CLASSE VALENDO NOTA**.

A implementação será realizada na Linguagem “C”.

DETALHAMENTO DO PROGRAMA: Banco ACME Master-Chef (Gestão do Banco)

Resumo: Fazer um programa com um menu de opções para a gestão das contas do BANCO ACME aplicativo Master-Chef - Contas dos usuários da Filial de Brasília. As funções do aplicativo são:

- 1 – Carregar o cadastro de contas de usuários com os respectivos saldos – Arquivo texto: “master.txt”.
- 2 – Realizar um depósito na conta selecionada, atualizando o saldo atual.
- 3 – Realizar um saque na conta selecionada, atualizando o saldo atual.
- 4 – Transferir saldo entre contas, troca saldo de uma para outra conta (“swap”), atualizando os saldos.
- 5 – Exibir os dados de uma determinada conta: nro. da conta, nome do correntista, saldo atual.
- 6 – Exibir dados de todas as contas cadastradas no banco: nro. conta, nome, saldo atual.
- 7 – Salvar o cadastro de contas de usuários com os respectivos saldos atuais – Arq. Texto: “final.txt”.
- 0 – Terminar as operações. Sair do aplicativo.

Importante, o programa deve possuir implementadas as rotinas conforme declaradas pelo professor e implementadas exatamente conforme a descrição fornecida. Estas rotinas serão usadas nas operações do Banco ACME: Programa MODULAR, com sub-rotinas, passagem de parâmetros, vetor e alocação dinâmica de memória, usando somente comandos de entrada de dados através do FGETS (sem usar comandos scanf).

A seguir serão detalhados os dados que tem que ser armazenados sobre as contas bancárias, os formatos dos arquivos textos de entrada (“master.txt”) e de saída de dados (“final.txt”), e como cada uma das opções do menu irá funcionar.

DADOS DAS CONTAS BANCÁRIAS:

- Número da conta: 6 dígitos. Exemplo: 123456 é uma conta do Banco ACME.
O banco não possui dígito de controle e nem senhas, pois é tudo conta “em família” (confiança total!)
- Nome do correntista: nome com até 30 caracteres (pode ter espaços). Exemplo: Senador Fulano
- Saldo Atual: Valor com 2 casas após a vírgula que representa o saldo (positivo ou negativo) da conta.

Estes dados devem estar juntos (agrupados) em uma Estrutura e em um Tipo de Dados próprio.

DADOS E EXEMPLO DE ARQUIVO TEXTO DE ENTRADA – “master.txt” (mesmo formato do arq. de saída)

Campos de dados: 1 dado em cada linha. Total na primeira linha seguido de conjuntos de 3 linhas com os dados de cada conta. O arquivo termina com a marca de fim “#” que indica que não tem mais dados.

<nro. “N” total de clientes>

```
<Nro. da conta do Cliente1>
<Nome do Cliente1>
<Saldo Atual do Cliente1>
< Nro. da conta do Cliente2>
<Nome do Cliente2>
<Saldo Atual do Cliente2>
...
<Nro. da conta do ClienteN>
<Nome do ClienteN>
<Saldo Atual do ClienteN>
#
```

Exemplo:

```
3
123456
Fulano da Silva
0.00
654321
Fulana da Silva e Silva
171.71
777888
Senador Fulano Jr.
10000000.00
#
```

SUB-ROTINAS A SEREM IMPLEMENTADAS:

DECLARAÇÃO E DOCUMENTAÇÃO DAS SUB-ROTINAS USADAS NO PROGRAMA

```
// Rotina le_menu_opcao: Ler do teclado (fgets) a opção numérica 0 a 7
// Entrada: nada # Saída: nro. da opção digitada
int le_menu_opcao(void);

// Rotina le_arquivo: Le o arquivo texto de dados com os dados dos clientes
// Entrada: nome do arquivo de entrada
// Saída: -1 = erro, ou, nro de clientes lidos (quantidade de contas)
// Atenção: o nome é sem o fim de linha ou caracteres invisíveis/inválidos
// Atenção: a rotina le_arquivo já aloca dinamicamente o vetor (cria cadastro).
int le_arquivo(char *nome);
```

```

// Rotorna grava_arquivo: Grava o arquivo texto de dados dos clientes no mesmo
// formato usado no arquivo de entrada
// Entrada: nome do arquivo e quantidade de clientes que existe no cadastro
// Saída: -1 = erro, ou, nro. de contas gravadas no arquivo
int grava_arquivo(char *nome,int nclientes);

// Rotina procura_conta: O usuário informa um nro. de conta e procura se tem
// no cadastro esta conta (nro. do cliente = índice do vetor/cadastro
// Entrada: nro da conta do cliente
// Saída: nro do cliente, ou seja, nro do índice do vetor do cadastro do cliente
// -1 = erro, se não conseguiu encontrar a conta no cadastro
int procura_conta(int conta);

// Rotina deposito: Adiciona um valor ao saldo do cliente
// Entrada: nro. do cliente (já procurou a conta) e valor do depósito
// Saída: valor do saldo atualizado após ter feito o depósito
double deposito(int ncliente, double valor);

// Rotina saque: Retira do saldo da conta o valor do saque
// Entrada: nro. do cliente (já procurou a conta) e valor do saque
// Saída: valor do saldo atualizado após ter feito o saque
double saque(int ncliente, double valor);

// Rotina transfere: Troca o saldo entre 2 contas, usando ponteiros,
// onde o valor do saldo da conta1 é trocado com o valor do saldo da conta2
// Entrada: ponteiro para a variável do saldo da conta1 e
// ponteiro para a variável do saldo da conta2
// Saída: (void) Saldos entre as contas são trocados no vetor/cadastro
void transfere(double *saldo1, double *saldo2);

// Rotina exhibe_saldo_conta: Exibe os dados de uma determinada conta
// Busca a conta do cliente e exibe os dados (nro. conta, nome, saldo)
// Entrada: nro da conta do cliente (tem que buscar no cadastro)
// Saída: (void) Exibe na tela os dados da conta (nro, nome, saldo)
void exhibe_saldo_conta(int conta);

// Rotina exhibe_contas: Exibe a lista de todas as contas cadastradas,
// Exibindo os dados do nro. da conta, nome do cliente e saldo atual
// Entrada: SEM ENTRADA (void)
// Saída: (void) Exibe na tela todas os dados das contas cadastradas
void exhibe_contas(void);

```

Outras sub-rotinas podem se implementadas se você achar necessário.

DESCRIÇÃO DO FUNCIONAMENTO DO PROGRAMA: OPÇÕES DO MENU

Laço que vai ler opções com a rotina “**le_menu_opção**”, executando as opções do menu, até que o usuário digite a opção 0 (sair) quando encerra o programa. O programa vai sempre começar lendo os dados do arquivo do cadastro de clientes (opção 1), e sempre terminar pela opção de sair/fim (opção 0).

1 – Ler Arquivo:

Lê o arquivo texto “master.txt”, cria o cadastro das contas no vetor alocado dinamicamente em memória, usando a sub-rotina “**le_arquivo**”. Usar o comando fgets para leituras do teclado e do arquivo (sem scanf/fscanf). Lembre-se de guardar apenas o nome do cliente, eliminando caracteres especiais/invisíveis (‘\n’, ‘\r’). Esta opção exibe na tela a mensagem “ERRO ARQUIVO” se deu erro na leitura, ou, “LIDOS <nro>”, indicando quantos clientes foram lidos para o cadastro (vetor de clientes).

2 – Depositar:

O usuário informa pelo teclado (fgets) qual é o número da conta onde deseja fazer o depósito, seguido do valor que deseja depositar nesta conta. O programa deve procurar pela conta, usando a sub-rotina “**procura_conta**”, se não achar a conta exibir na tela a mensagem “ERRO CONTA”, e se achar no cadastro, atualizar o saldo desta

conta no cadastro, adicionando o valor a ser depositado no saldo, exibindo na tela a mensagem “SALDO <valor>” com o valor sendo exibido com 2 casas após a vírgula. Para adicionar ao saldo use a sub-rotina “**deposito**”.

3 – Sacar:

O usuário informa pelo teclado (fgets) qual é o número da conta onde deseja fazer o saque, seguido do valor que deseja sacar desta conta. O programa deve procurar pela conta, usando a sub-rotina “**procura_conta**”, se não achar a conta exibir na tela a mensagem “ERRO CONTA”, e se achar no cadastro, atualizar o saldo desta conta no cadastro, subtraindo o valor a ser sacado do saldo, exibindo na tela a mensagem “SALDO <valor>” com o valor sendo exibido com 2 casas após a vírgula. Para sacar do saldo use a sub-rotina “**saque**”.

4 – Transferir Saldos:

O usuário informa pelo teclado (fgets) qual é o número da conta1 cujo saldo será trocado, seguido do número da conta2 cujo saldo será trocado. O programa deve procurar pela conta1 e pela conta2, usando a sub-rotina “**procura_conta**”, se não achar alguma destas duas contas, exibir na tela a mensagem “ERRO CONTA1” ou “ERRO CONTA2”, respectivamente. Se achar ambas as contas no cadastro, trocar o saldo da conta1 com o saldo da conta2 (troca/transferência de saldos entre contas), usando a sub-rotina “**transfere**”, e exibindo na tela a mensagem de “TRANSF OK” para indicar que a transferência foi realizada.

5 – Consulta Saldo de uma Conta:

O usuário informa pelo teclado (fgets) qual é o número da conta cujos dados devem ser exibidos na tela, onde o programa busca para ver se encontra esta conta no cadastro, usando a sub-rotina “**procura_conta**”. Se não encontrar a conta, exibe a mensagem “ERRO CONTA”, e se encontrar no cadastro, exibe os dados da conta na tela: nro. da conta, nome do cliente e saldo atual (com 2 casas após a vírgula), sendo um dado em cada linha da tela. Esta opção deve usar a sub-rotina “**exibe_saldo_contas**” para exibir na tela os dados da conta selecionada.

6 – Consulta Todas Contas:

Esta opção usa a sub-rotina “**exibe_contas**” para exibir os dados de TODAS as contas cadastradas. Inicialmente exibe na tela o número total de contas cadastradas “CONTAS <nro>” e depois segue exibindo os dados das contas nas linhas abaixo (3 linhas por conta), contendo o número da conta, o nome do cliente e o saldo (2 casas decimais).

7 – Salvar Arquivo:

Escreve o arquivo texto “final.txt”, salvando em disco o cadastro das contas que está armazenado no vetor alocado em memória, usando a sub-rotina “**grava_arquivo**”. O formato do arquivo de saída é o mesmo do arq. de entrada. Esta opção exibe na tela a mensagem “ERRO ARQUIVO” se deu erro na gravação, ou, “GRAVAOD <nro>”, indicando quantos clientes foram gravados no arquivo em disco.

0 – Sair do Programa (FIM):

O programa termina a execução quando o usuário digitar a opção 0 (FIM).

BOM TRABALHO!

Documento: Trabalho Extra-Classe EX02 - Versão 1.0 Rev. 00 de 19/06/2026

=====

F.Osório
Junho 2026

=====